

Cleft - เอกสารสรุปโปรเจกต์ฉบับละเอียด

อัปเดตสถานะระบบ: 2 กันยายน 2026

สถานะล่าสุด: Frontend, Backend, PostgreSQL, Prisma migration และ Docker Compose ผ่านการทดสอบแบบ end-to-end แล้ว

1. ภาพรวมโปรเจกต์

Cleft คือเว็บแอปสำหรับแบ่งค่าใช้จ่ายของกลุ่ม เช่น การรับประทานอาหาร งานเลี้ยง หรือกิจกรรมร่วมกัน ผู้ใช้สามารถเพิ่มรายการและจำนวนสินค้า เพิ่มสมาชิก กำหนดว่าสมาชิกแต่ละคนร่วมจ่ายทุกรายการหรือเฉพาะรายการที่เลือก ตรวจสอบยอดของแต่ละคน และยืนยันบิลเพื่อบันทึกประวัติลงฐานข้อมูล

ระบบออกแบบเป็น Full Stack แยก Frontend, Backend และ Database ออกจากกันอย่างชัดเจน โดยมีเป้าหมายหลักดังนี้:

- ลดความผิดพลาดจากการคำนวณด้วยมือ
- รองรับรายการที่มีผู้รับผิดชอบไม่เหมือนกัน
- แสดงยอดที่สมาชิกแต่ละคนต้องจ่ายอย่างโปร่งใส
- เก็บ draft ในเครื่องก่อนยืนยัน เพื่อไม่ให้ข้อมูลที่กำลังกรอกสูญหาย
- บันทึกบิลที่ยืนยันแล้วและเรียกดูประวัติย้อนหลังได้
- รองรับการรันแบบ local และ Docker Compose

สิ่งสำคัญที่ต้องเข้าใจคือ Cleft เป็นระบบคำนวณและบันทึกการแบ่งบิล ไม่ใช่ระบบโอนเงินจริง และในโมเดลปัจจุบันยังไม่มีฟีเจอร์เก็บสถานะว่าใครชำระเงินแล้วหรือใครเป็นผู้สำรองจ่าย

2. ขอบเขตของระบบ

2.1 ความสามารถที่มีอยู่

- เข้าสู่ระบบด้วย Google OAuth 2.0
- ใช้ JWT access token และ refresh token ผ่าน HTTP-only cookies
- อ่านและแก้ไขชื่อผู้ใช้ รูป avatar และสัญลักษณ์สกุลเงิน
- เพิ่ม แก้ไข และลบรายการค่าใช้จ่าย
- ระบุราคา จำนวน และหมายเหตุของแต่ละรายการ
- เพิ่ม แก้ไข และลบสมาชิก
- แบ่งค่าใช้จ่ายแบบ ALL และ PARTIAL
- คำนวณยอดต่อคนใน Frontend เพื่อแสดงผลทันที
- คำนวณซ้ำใน Backend สำหรับ endpoint ที่รับข้อมูลการคำนวณ
- แสดงหน้า Summary พร้อมสัดส่วนของสมาชิกแต่ละคน

- แบนรูปสลิปประเภท JPG, JPEG, PNG, GIF หรือ WebP ขนาดไม่เกิน 5 MB
- ยืนยันและบันทึก Party, Item, Participant, Consumption และ History
- ดูประวัติบิลล่าสุดหรือประวัติทั้งหมด
- เก็บ draft ที่ยังไม่ยืนยันใน `localStorage`
- รันระบบด้วย Docker Compose พร้อม healthcheck และ migration อัตโนมัติ

2.2 สิ่งที่ยังอยู่นอกขอบเขต

- การโอนเงินจริงผ่านธนาคารหรือ e-wallet
- การติดตามสถานะ paid/unpaid
- การระบุผู้สำรองจ่ายหรือเจ้าหน้าที่ของแต่ละรายการ
- ภาษี ค่าบริการ ส่วนลด หรือ tip แบบแยกช่อง
- อัตราแลกเปลี่ยนหลายสกุลเงิน
- การแก้ไขบิลที่ยืนยันแล้ว
- การทำงานร่วมกันหลายคนแบบ real-time
- ระบบแจ้งเตือนผ่าน Email, LINE หรือ SMS
- Admin panel และระบบกำหนดสิทธิ์หลายระดับ

3. เทคโนโลยีและหน้าที่ของแต่ละส่วน

ชั้นระบบ	เทคโนโลยี	หน้าที่
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS	UI, routes, draft session, API client และ visualization
Client data	TanStack Query, Axios	cache request, session/profile/history และ refresh token flow
Charts	Recharts	กราฟการกระจายยอดค่าใช้จ่าย
Backend	NestJS 12, TypeScript	REST API, authentication, validation และ business logic
Authentication	Passport, Google OAuth, JWT	login, session, refresh และ logout
ORM	Prisma 5	schema, migration, query และ transaction
Database	PostgreSQL 16	จัดเก็บข้อมูลถาวร
File upload	Multer	ตรวจชนิด ขนาด และบันทึกรูปสลิป
Container	Docker, Docker Compose	รัน Frontend, Backend และ PostgreSQL ร่วมกัน
Quality	TypeScript, ESLint, smoke test	ตรวจ build, lint และ flow การใช้งานจริง

4. โครงสร้างโฟลเดอร์

```
Project_github/
|-- frontend/                Next.js application
|   |-- app/                 Home, Items, Members, Summary, Dashboard
|   |-- components/         Navigation, profile, charts และ UI primitives
|   |-- hooks/              local party session
|   |-- lib/                API client, types และ calculation helpers
|   |-- public/             icons และ PWA assets
|   |-- Dockerfile
|   `-- .env.example
|-- backend/                 NestJS application
|   |-- src/auth/            Google OAuth, JWT, refresh และ logout
|   |-- src/users/           profile API
|   |-- src/party/           calculate, confirm และ history API
|   |-- src/prisma/          Prisma service
|   |-- src/config/          environment validation
|   |-- prisma/              schema และ migrations
|   |-- scripts/             automated smoke test
|   |-- Dockerfile
|   `-- .env.example
|-- uploads/                 รูปสลิปที่อัปโหลดใน local/Docker
|-- scripts/                 utility scripts ของโปรเจกต์
|-- docker-compose.yml       local full-stack orchestration
|-- Dockerfile               backend image สำหรับ host ที่ build จาก repo root
|-- README.md                วิธีรันและ deploy
|-- DATAFLOW.md             เอกสาร data flow
|-- SUMMARY.md               source ของเอกสารฉบับนี้
`-- SUMMARY.pdf              เอกสาร PDF ที่สร้างจาก SUMMARY.md
```

ไฟล์ `.env` จริงถูกละเว้นจาก Git ด้วย `.gitignore` เพื่อไม่ให้ credentials และ JWT secrets ถูกอัปโหลดขึ้น repository โดยจะอัปโหลดเฉพาะ `.env.example`

5. สถาปัตยกรรมและ Data Pipeline

```
Browser
|
| เปิดหน้า /, /items, /members, /summary
v
Next.js Frontend :3000
|-- เก็บ draft ใน localStorage
|-- แสดงผลการคำนวณทันที
|-- เรียก same-origin /api/*
v
Next.js Rewrite /api/*
|-- local -> http://localhost:4000
|-- Docker -> http://backend:4000
v
NestJS Backend :4000
|-- Helmet / CORS / Cookie Parser
|-- Passport JWT Guard
|-- ValidationPipe และ DTO validation
|-- Party service / User service / Auth service
v
Prisma ORM
|-- migration deploy ตอน container เริ่มทำงาน
|-- transaction ตอนยืนยันบิล
v
PostgreSQL :5432
```

Frontend ใช้ URL `/api` เสมอ จึงไม่เปิดเผย Backend URL ให้ client code และช่วยให้ cookies อยู่ใน origin เดียวกับหน้าเว็บ ส่วนปลายทางของ rewrite กำหนดด้วย `API_PROXY_TARGET` ในช่วง build ของ Frontend

6. เส้นทางการใช้งานของผู้ใช้

6.1 Login

1. ผู้ใช้กด `Continue with Google` ที่หน้า Home
2. Browser ไปที่ `/api/auth/google`
3. Next.js proxy request ไป Backend
4. Passport เปลี่ยนเส้นทางไป Google OAuth
5. Google เรียก callback ที่ Frontend URL เช่น `/api/auth/google/callback`
6. Next.js proxy callback ไป Backend
7. Backend สร้างหรืออัปเดต User ด้วย Prisma
8. Backend สร้าง access token และ refresh token
9. Token ถูกเก็บใน HTTP-only cookies และผู้ใช้ถูกส่งกลับหน้า Home

การให้ callback กลับผ่าน Frontend มีความสำคัญ เพราะทำให้ cookie ถูกกำหนดบนโดเมนเดียวกับแอป แม้ Frontend และ Backend จะ deploy แยก service

6.2 สร้างและยืนยันบิล

1. หน้า Items เพิ่มชื่อ ราคา จำนวน และหมายเหตุ
2. ราคา line total คำนวณจาก $\text{price} \times \text{quantity}$
3. หน้า Members เพิ่มสมาชิกและเลือก ALL หรือ PARTIAL
4. ถ้าเป็น PARTIAL ผู้ใช้เลือกรายการที่สมาชิกคนนั้นร่วมจ่าย
5. Draft ทั้งหมดบันทึกใน localStorage ภายใต้อัก key cleft-session
6. หน้า Summary คำนวณยอดและแสดงสัดส่วนต่อสมาชิก
7. เมื่อกด Confirm ระบบสร้าง multipart form ซึ่งประกอบด้วย JSON payload และรูปสลิปถ้ามี
8. Backend ตรวจ auth, parse JSON, validate DTO และตรวจชื่อซ้ำ/รายการที่ไม่รู้จัก
9. Prisma transaction สร้าง Party, Items, Participants, Consumptions และ History
10. เมื่อสำเร็จ Frontend ล้างรายการและสมาชิกของ draft แล้วกลับหน้า Home

7. กติการคำนวณค่าใช้จ่าย

7.1 ความหมายของ Split Type

- ALL : สมาชิกถูกนำไปหารในทุกรายการ
- PARTIAL : สมาชิกถูกนำไปหารเฉพาะรายการที่เลือกใน itemNames
- PARTIAL ที่ไม่เลือกรายการใดมียอดเป็น 0

7.2 สูตรต่อรายการ

สำหรับแต่ละ item:

```
lineTotal = price x quantity
assignees = สมาชิก ALL + สมาชิก PARTIAL ที่เลือก item นี้
sharePerPerson = lineTotal / จำนวน assignees
ยอดสมาชิก = ผลรวม sharePerPerson ของทุกรายการที่สมาชิกได้รับมอบหมาย
```

ถ้ารายการหนึ่งไม่มี assignee ระบบจะไม่กระจายยอดของรายการนั้นให้ใคร แต่ยอดรวมของบิลยังรวมราคาของรายการดังกล่าวอยู่ ผู้ใช้จึงควรตรวจให้แน่ใจว่ามีสมาชิก ALL อย่างน้อยหนึ่งคน หรือกำหนดสมาชิก PARTIAL ให้ครบทุกรายการ

7.3 ตัวอย่างที่ผ่าน smoke test

รายการ	ราคา	ผู้ร่วมจ่าย	ผลลัพธ์
Pizza	300	Alice (ALL), Bob (PARTIAL)	คนละ 150

รายการ	ราคา	ผู้ร่วมจ่าย	ผลลัพธ์
Drink	100	Alice (ALL)	Alice 100

ผลรวมคือ 400 บาท: Alice จ่าย 250 บาท และ Bob จ่าย 150 บาท

Backend ปิดยอดสุดท้ายของสมาชิกแต่ละคนให้เหลือ 2 ตำแหน่ง การหารที่ไม่ลงตัวอาจทำให้ผลรวมของยอดที่ปิดแล้วแตกต่างจากยอดบิลเล็กน้อยในระดับสตางค์ ระบบปัจจุบันยังไม่มีขั้นตอนแจกจ่ายเศษจากการปิด

8. REST API

Method	Endpoint	Auth	หน้าที่
GET	/health	ไม่ต้อง	ตรวจว่า Backend พร้อมทำงาน
GET	/auth/google	ไม่ต้อง	เริ่ม Google OAuth
GET	/auth/google/callback	Google guard	รับผล OAuth และตั้ง cookies
GET	/auth/session	JWT	ตรวจ session ปัจจุบัน
POST	/auth/refresh	Refresh cookie	หมุน refresh token และออก access token ใหม่
POST	/auth/logout	JWT	revoke refresh token และล้าง cookies
GET	/users/profile	JWT	อ่านข้อมูลผู้ใช้
PATCH	/users/profile	JWT	แก้ username, avatar หรือ currency symbol
POST	/party/calculate	JWT	validate และคำนวณยอดแบ่งบิล
POST	/party/confirm	JWT	บันทึกบิลและสลิปด้วย transaction
GET	/party/history	JWT	อ่านประวัติล่าสุด 3 รายการ
GET	/party/history? all=true	JWT	อ่านประวัติทั้งหมด

เมื่อ access token หมดอายุ Axios interceptor จะเรียก /auth/refresh หนึ่งครั้ง คิว request ที่ได้รับ 401 ระหว่าง refresh จะรอผลเดียวกัน เพื่อลดการยิง refresh ซ้ำพร้อมกัน

9. Authentication และ Security

9.1 Token lifecycle

- Access token: ค่าเริ่มต้น 15 นาที
- Refresh token: ค่าเริ่มต้น 7 วัน
- Refresh token ตัวจริงเก็บใน HTTP-only cookie
- Database เก็บเฉพาะ SHA-256 hash ของ refresh token
- การ refresh จะตรวจ hash เดิมและหมุน refresh token ใหม่
- Logout ลบ hash ในฐานข้อมูลและล้าง cookies

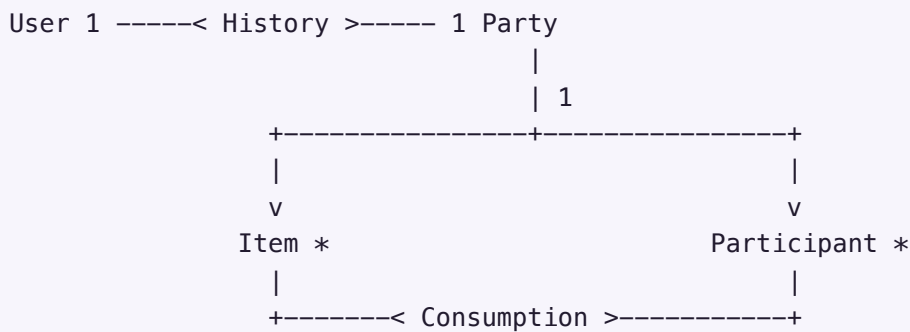
9.2 มาตรการที่มีอยู่

- Helmet เพิ่ม HTTP security headers
- CORS จำกัด origin ตาม `FRONTEND_URL`
- Cookies เป็น HTTP-only และเป็น Secure ใน production
- JWT Guard ป้องกัน profile, calculate, confirm และ history
- Global `ValidationPipe` เปิด whitelist และ transform
- Confirm payload เปิด `forbidNonWhitelisted`
- อัปโหลดสลิปจำกัด 5 MB และตรวจทั้ง MIME type กับ extension
- Environment validation ตรวจ required variables, port, cookie mode และความยาว JWT secret ใน production
- `.env` และ uploads ถูกละเว้นจาก Git

9.3 ข้อควรทำก่อน production

- สร้าง JWT secrets แบบสุ่มและยาวอย่างน้อย 32 ตัวอักษร
- ใช้ Google Client ID/Secret จริง
- ตั้ง Authorized redirect URI ให้ตรงกับ `GOOGLE_CALLBACK_URL` ทุกตัวอักษร
- ใช้ HTTPS ทั้ง Frontend และ Backend
- ใช้ object storage สำหรับสลิป ถ้า host มี filesystem ชั่วคราว
- จำกัดสิทธิ์และหมุน database credentials เป็นระยะ

10. Database และ ER Model



10.1 Model

User

- เก็บ Google identity, email, username, avatar และ currency symbol
- `googleId` และ `email` เป็น unique
- เก็บ `refreshTokenHash` สำหรับ refresh token rotation

Party

- เก็บชื่อ วันที่ ยอดรวม และ URL ของสลิป
- เป็น parent ของ Item และ Participant

Item

- เก็บชื่อ ราคา และหมายเหตุของรายการที่ยืนยันแล้ว
- จำนวนสินค้าถูกรวมเข้า price ก่อนบันทึก และข้อความ quantity ถูกแนบใน note

Participant

- เก็บชื่อและ split type ของสมาชิกใน Party

Consumption

- ตาราง many-to-many ระหว่าง Participant กับ Item
- composite primary key คือ `participantId + itemId`
- ใช้บันทึก item selection ของสมาชิก `PARTIAL`

History

- เชื่อม User กับ Party ที่ผู้ใช้นั้นยืนยัน
- ใช้สำหรับหน้า Recent History

Relation สำคัญตั้งค่า `onDelete: Cascade` เพื่อป้องกัน record ลูกกำพร้าเมื่อ record หลักถูกลบ

11. Local Draft และข้อมูลถาวร

ระบบมีข้อมูลสองช่วงชีวิต:

1. **Draft data** อยู่ใน browser `localStorage` และแก้ไขได้ทันที
2. **Confirmed data** อยู่ใน PostgreSQL และอ่านผ่าน History API

Draft ประกอบด้วย:

```
partyName
partyDate
items[]
members[]
```

การ logout จะลบ `cleft-session` ออกจาก `localStorage` ส่วนการ confirm สำเร็จจะเก็บชื่อ Party ไว้ แต่รีเซ็ตวันที่ รายการ และสมาชิก

ข้อจำกัดคือ draft ไม่ sync ระหว่างอุปกรณ์ และข้อมูลอาจสูญหายเมื่อผู้ใช้ล้าง browser storage

12. Docker และการเริ่มระบบ

Docker Compose มี 3 services:

Service	Port	Health condition
PostgreSQL	5432	<code>pg_isready</code>
Backend	4000	<code>GET /health</code>
Frontend	3000	เริ่มหลัง Backend healthy

ลำดับการเริ่มระบบ:

```
PostgreSQL healthy
-> Backend รัน prisma migrate deploy
-> Backend เริ่ม NestJS และ healthcheck ผ่าน
-> Frontend เริ่ม Next.js
```

สำหรับ Docker ค่า `API_PROXY_TARGET` ต้องเป็น `http://backend:4000` และถูกส่งเป็น build argument เพราะ Next.js สร้าง rewrite ระหว่าง build ส่วนการรัน Frontend โดยตรงในเครื่องใช้ `http://localhost:4000`

คำสั่งหลัก:

```
docker compose up -d --build
docker compose ps
docker compose logs -f backend
docker compose down
```

13. Environment Variables

13.1 Backend

Variable	ความหมาย
DATABASE_URL	PostgreSQL connection string
PORT	Backend port ค่าเริ่มต้น 4000
FRONTEND_URL	origin ที่อนุญาตและปลายทางหลัง login
GOOGLE_CLIENT_ID	Google OAuth client ID
GOOGLE_CLIENT_SECRET	Google OAuth client secret
GOOGLE_CALLBACK_URL	callback ผ่าน Frontend <code>/api/auth/google/callback</code>
JWT_ACCESS_SECRET	secret สำหรับ access token
JWT_REFRESH_SECRET	secret สำหรับ refresh token
JWT_ACCESS_TTL	อายุ access token
JWT_REFRESH_TTL	อายุ refresh token
COOKIE_SAME_SITE	lax หรือ none

13.2 Frontend

Variable	ความหมาย
API_PROXY_TARGET	Backend URL ที่ Next.js ใช้สร้าง <code>/api</code> rewrite

ห้าม commit `.env` จริง ให้คัดลอกจาก `.env.example` และตั้งค่าผ่าน secret manager ของ hosting platform

14. Validation และ Error Handling

14.1 Frontend

- ไม่อนุญาต item name หรือ member name ซ้ำโดยไม่สนตัวพิมพ์ใหญ่เล็ก
- ราคาและ quantity ต้องเป็นตัวเลขที่ถูกต้อง ราคาเป็นบวก และ quantity อย่างน้อย 1
- เมื่อเปลี่ยนชื่อ item จะอัปเดต references ในสมาชิก **PARTIAL**
- เมื่อลบ item จะลบ references จากสมาชิกทุกคน
- ปุ่ม Confirm ถูกปิดถ้าขาด party name, items หรือ members
- Toast แสดงผล create, update, delete, upload และ confirm

14.2 Backend

- ชื่อ item และ participant ต้องไม่ว่างและไม่ซ้ำ
- **itemNames** ของสมาชิกห้ามอ้างอิงถึง item ที่ไม่มีอยู่
- DTO ตรวจสอบข้อมูล, positive price, date และ nested arrays
- Confirm ใช้ transaction เพื่อไม่ให้ข้อมูลบันทึกเพียงบางส่วน
- File filter ปฏิเสธชนิดและ extension ที่ไม่อนุญาต
- Environment ที่ไม่ครบทำให้ Backend หยุดพร้อมข้อความที่ระบุ key ที่ขาด

15. การทดสอบและสถานะล่าสุด

การตรวจล่าสุดครอบคลุม:

การตรวจ	ผลลัพธ์
Frontend TypeScript	ผ่าน
Frontend production build	ผ่านครบทุก route
Backend TypeScript	ผ่าน
Backend ESLint 9 flat config	ผ่าน
Backend production build	ผ่าน
Prisma schema validation	ผ่าน
Initial migration	ลง PostgreSQL สำเร็จ
Docker Compose config	ผ่าน
PostgreSQL healthcheck	healthy
Backend healthcheck	healthy

การตรวจ	ผลลัพธ์
Frontend root request	HTTP 200
Frontend /api/health proxy	HTTP 200
Guest /api/auth/session	HTTP 401 ตามที่คาด
OAuth start route	HTTP 302 ไป Google พร้อม callback ที่ถูกต้อง
npm dependency audit	0 vulnerabilities หลังอัปเดต lockfile

Smoke test สร้างข้อมูลชั่วคราวและตรวจ flow ต่อไปนี้:

```
Health
-> JWT authenticated session
-> Update profile
-> Calculate ALL/PARTIAL shares
-> Confirm party ด้วย multipart payload
-> Read confirmed party from history
-> Delete test data
```

ผลลัพธ์คือ Smoke test passed: health, auth, profile, calculation, confirm, and history

Google OAuth callback จริงไม่สามารถทำอัตโนมัติได้จนกว่าจะใส่ Google Client ID/Secret ของเจ้าของโปรเจกต์ แต่ routing และ redirect URL ถูกตรวจแล้ว

16. จุดผิดพลาดที่พบบ่อยและวิธีป้องกัน

Backend build ผ่านแต่เริ่มระบบไม่ได้

สาเหตุเดิมคือโมดูลที่ใช้ JwtAuthGuard ไม่มี AuthModuleOptions ของ Passport วิธีป้องกันคือ import PassportModule.register({ defaultStrategy: "jwt" }) ในทุก feature module ที่ใช้ guard

Docker Compose เริ่มไม่ได้เพราะไม่มี .env

ต้องสร้าง backend/.env จาก backend/.env.example ก่อน ส่วน Frontend Docker ได้รับ proxy target จาก build argument ใน Compose

Database เชื่อมต่อได้แต่ไม่มีตาราง

ต้อง commit Prisma migrations และใช้ prisma migrate deploy ตอน Backend container เริ่มทำงาน ห้ามพึ่ง prisma db push ใน production

Frontend container เรียก localhost:4000

ภายใน container คำว่า localhost หมายถึง container ตัวเอง ต้องใช้ service name `http://backend:4000` และส่งค่าในช่วง build

Login สำเร็จที่ Backend แต่ Frontend ไม่มี cookie

เมื่อ deploy แยกโดเมน callback ควรกลับผ่าน Frontend `/api/auth/google/callback` เพื่อให้ Set-Cookie อยู่ใน origin เดียวกับแอป

ESLint 9 หา config ไม่พบ

ESLint 9 ใช้ flat config จึงต้องมี `eslint.config.mjs` และ TypeScript parser/config ที่เข้ากันได้

เปลี่ยน .env หลัง Frontend build แต่ rewrite ไม่เปลี่ยน

`API_PROXY_TARGET` ถูกอ่านใน `next.config.mjs` ระหว่าง build จึงต้อง rebuild Frontend image เมื่อเปลี่ยน Backend URL

17. ข้อจำกัดและงานต่อ ยอดที่แนะนำ

ความสำคัญสูง

- เพิ่ม payer model เพื่อระบุว่าใครสำรองจ่าย และคำนวณยอดรับคืนจริง
- เพิ่ม paid/unpaid state และ settlement history
- เพิ่ม integration test สำหรับ refresh cookie rotation และ logout
- ย้าย slip upload ไป object storage สำหรับ production
- เพิ่ม rate limiting โดยเฉพาะ auth และ upload endpoints

ความสำคัญปานกลาง

- เก็บ quantity เป็น field ในฐานข้อมูลแทนการรวมเข้า price/note
- เพิ่ม tax, service charge, discount และ tip
- จัดการเศษสตางค์จากการปัดยอดอย่าง deterministic
- เพิ่ม error boundary และหน้า error เฉพาะ route
- เพิ่ม pagination สำหรับ history จำนวนมาก

UX และผลิตภัณฑ์

- sync draft ข้ามอุปกรณ์
- แชร์บิลด้วย link หรือ QR code
- export summary เป็นรูปหรือ PDF
- รองรับหลายภาษาและหลายสกุลเงิน
- เพิ่ม accessibility test และ automated browser test

18. สรุป

Cleft มีโครงสร้าง Full Stack ที่ชัดเจนและครอบคลุม flow หลักตั้งแต่ authentication, draft bill, item/member assignment, calculation, confirmation, file upload และ history ระบบใช้ same-origin API proxy เพื่อให้ Frontend และ Backend ทำงานร่วมกันได้ทั้ง local, Docker และ deployment แบบแยก service

หลังการปรับปรุงล่าสุด จุดบดล็อกการรันถูกแก้แล้ว ได้แก่ Passport module configuration, environment validation, Prisma migration, Docker service health/order, API proxy target, OAuth callback routing, ESLint และ automated smoke test

สถานะปัจจุบันพร้อมสำหรับการพัฒนาต่อและ deploy หลังจากเจ้าของโปรเจกต์ใส่ credentials จริง ได้แก่ Google OAuth, production PostgreSQL URL และ JWT secrets ที่ปลอดภัย